



1

カードゲーム 4 (Card Game 4)

Author : 蜂矢 倫久 (Mitsubachi)

各小課題について，Python における実装例を記した．実装において参考にとすると良い．

小課題 1

$N = K$ の場合においては，カードの選び方はすべてのカードを選ぶ場合のみである．

したがって， A_i がすべて偶数またはすべて奇数ならば答えは A_i の総和であり，そうでない場合，答えは 0 である．

A_i がすべて偶数であるかなどは for 文や if 文を用いて調べることができる． A_i の偶奇がすべて同じということを， A_i の偶奇が A_1 の偶奇に等しいと言い換えることでも正解することができる．

```
1  n, k = map(int, input().split())
2  a = list(map(int, input().split()))
3
4  sum = 0
5
6  for i in range(n):
7      sum += a[i]
8      if a[i] % 2 != a[0] % 2:
9          print(0)
10         exit()
11
12 print(sum)
```



小課題 2

A_i としてありうるのは 1 または 2 である。よって、得点が 0 でないとき、1 が書かれたカードのみ、もしくは 2 が書かれたカードのみを選んでいる。

したがって、1 が書かれたカードが K 枚以上あるならば得点を K とすることができる。また、2 が書かれたカードが K 枚以上あるならば得点を $2K$ とすることができる。

1 が書かれたカードが K 枚以上あるかなどは for 文や if 文を用いて調べることができる。

```
1  n, k = map(int, input().split())
2  a = list(map(int, input().split()))
3
4  one = 0
5  two = 0
6
7  for i in range(n):
8      if a[i] == 1:
9          one += 1
10     else:
11         two += 1
12
13  ans = 0
14
15  if one >= k:
16      ans = k
17  if two >= k:
18      ans = 2 * k
19
20  print(ans)
```

小課題 3

A_i はすべて奇数であるため、どのように選んでも得点は選んだカードに書かれた A_i の総和となる。

したがって、 A_i の大きい順に K 枚選ぶことが最適である。 A_i の大きい順に並び替えるには sort 関数を用いれば良い。



```
1  n, k = map(int, input().split())
2  a = list(map(int, input().split()))
3
4  a.sort(reverse = True)
5
6  ans = 0
7
8  for i in range(k):
9      ans += a[i]
10
11 print(ans)
```

小課題 4

A_i がすべて偶数である選び方とすべて奇数である選び方のみを考えれば良い。配列 B, C を A_i のうちそれぞれ偶数, 奇数であるものを集めたものとする。

B の長さが K 以上ならば A_i がすべて偶数であるように選ぶことができる。このときの得点としてありうる最大値は小課題 3 と同様に考えれば, B の要素を大きい順に K 個選べば良い。これは `sort` 関数により行うことができる。

C についても同様に行えば, すべて奇数である選び方の得点としてありうる最大値も求められる。

```
1  n, k = map(int, input().split())
2  a = list(map(int, input().split()))
3
4  odd = [ai for ai in a if ai % 2 != 0]
5  even = [ai for ai in a if ai % 2 == 0]
6
7  odd_sum = sum(sorted(odd, reverse=True)[:k]) if len(odd) >= k else 0
8  even_sum = sum(sorted(even, reverse=True)[:k]) if len(even) >= k else 0
9
10 print(max(odd_sum, even_sum))
```



2

ポスター 2 (Poster 2)

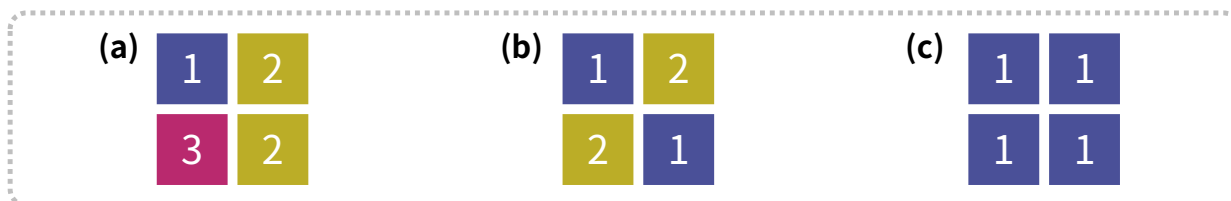
解説作成者：米田優峻

小課題 1 (累計 9 点)

制約： $N = 2, K = 3$

この小課題では、ポスターのマス目の大きさが 2×2 です。そこで、下図の (a), (b), (c) のケースにおける答えを考えてみましょう。

- 3 種類の色が出現する (a) のケースでは、元々からカラフル度が 1 であり、操作によってそれ以上増やすことができないため答えは 1 となります。
- 2 種類の色が出現する (b) のケースでは、右下の色 1 を色 3 に変えることでカラフル度を 1 にできるので、答えは 1 になります。
- 1 種類の色が出現する (c) のケースでは、どのような操作を行ってもカラフル度を 0 から増やすことができないため、答えは 0 となります。



このように考えていくと、答えは色が 1 種類しか出現しない場合 0，そうでない場合 1 であることが分かります。したがって、 $A_{1,1}, A_{1,2}, A_{2,1}, A_{2,2}$ が同じかどうかを if 文で判定すれば良いです*1。

小課題 2 (累計 15 点)

制約： $A_{i,j}$ はすべて異なる

マス目の大きさが $N \times N$ のとき、どんなマス目でも、カラフル度が $(N-1)^2$ を上回るとは絶対にあり得ません。一方、 $A_{i,j}$ がすべて異なる場合、操作を行わずとも、カラフル度 $(N-1)^2$ が達成されています。したがって、単に $(N-1)^2$ の値を出力すれば小課題 2 に正解します。たとえば C++ を使って実装する場合、`cout << (N-1) * (N-1) << endl;` とすれば良いです。

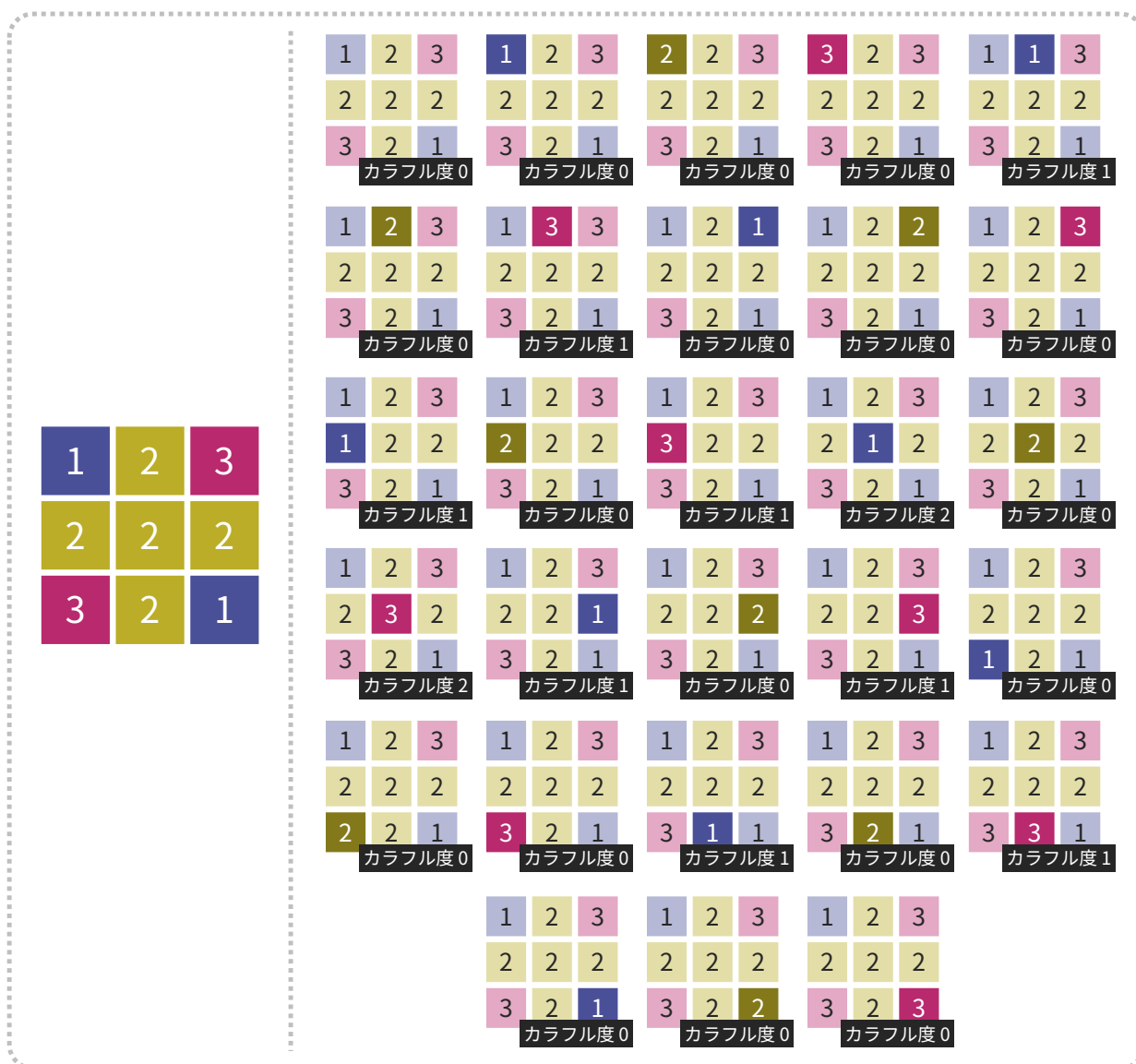
*1 情報オリンピックの Web サイトに掲載されているサンプルコードを参照のこと。



小課題 3 (累計 42 点)

制約: $N \leq 10, K \leq 10$

まずは、下図左にある入力例 4 を手で解くことを考えてみましょう。少し面倒な方法ですが、あり得る操作方法をすべて調べ上げ、それぞれについてカラフル度を計算すると、答えを求めることができます。入力例 4 の場合、操作方法は操作を 1 回もしない場合が 1 通り、操作を 1 回行う場合が $9 \times 3 = 27$ 通り (9 はマスの数, 3 は色の数) あるので、合計 28 通りを調べれば良いです。すると下図右のように、カラフル度の最大値として 2 が達成できると分かります。





実は、この小課題の制約は $N \leq 10, K \leq 10$ と小さいので、この方法でも小課題 3 に正解することができます。なお、一般に、あり得るすべてのパターンを調べる手法のことを**全探索**といいます。

問題は、どのようにしてプログラムを実装するかです*²。まず、色 a, b, c, d の中に 3 種類以上の色が含まれるかどうかを判定する関数は、以下のように実装することができます。*³

```
1 // a, b, c, d の中に 3 種類以上含まれるか？
2 bool check(int a, int b, int c, int d) {
3     // 3 つ同じ数がある
4     if (a == b && a == c) return false;
5     if (a == b && a == d) return false;
6     if (a == c && a == d) return false;
7     if (b == c && b == d) return false;
8
9     // 2 つ+ 2 つになっているケース
10    if (a == b && c == d) return false;
11    if (a == c && b == d) return false;
12    if (a == d && b == c) return false;
13
14    // それ以外はOK
15    return true;
16 }
```

すると、マス (x, y) の色を c に変えたときのカラフル度を求める関数は、以下のようにして実装することができます。ただし、二次元配列 A は最初のマス目の状態を表します。*⁴

```
1 // マス(x, y) の色をc に変えたときのカラフル度を返す
2 // N はマス目の大きさ、A は最初のマス目の情報を管理した配列
3 int get_colorful(int x, int y, int c, int N, vector<vector<int>> A) {
4     // マス(x, y) の色を変える
5     A[x][y] = c;
6
7     // コストを求める
8     int cost = 0;
9     for (int i = 1; i <= N - 1; i++) {
10         for (int j = 1; j <= N - 1; j++) {
11             // A[i][j], A[i][j+1], A[i+1][j], A[i+1][j+1]
```

*² 以下は C++ をベースに説明しますが、Python の場合も情報オリンピックのウェブサイトにサンプルコードが掲載されているので、ぜひ参考にしてください。

*³ 他にもいろいろな方法がありますが、今回は最も簡単な方法である、if 文の場合分けによる解法を取り上げています。

*⁴ `vector<vector<int>> A` のような書き方に慣れない方は、「C++ vector 型」と検索し、色々なプログラムを試してみましょう。



```
12             // その中に3 種類以上あれば、カラフル度を1 増やす
13             if (check(A[i][j],A[i][j+1],A[i+1][j],A[i+1][j+1])==true) {
14                 cost += 1;
15             }
16         }
17     }
18     return cost;
19 }
```

あとは以下のようにして入力を行い、三重の for 文ループを使ってあり得るすべての (x,y,c) を全探索すると、答えを求めることができます。

なお、1 回も操作しない場合が気になるかもしれませんが、 $(x,y) = (1,1)$ とし、 c を元々のマス $(1,1)$ の色とすると実質的に 1 回も操作しない場合と同じなので、問題は起こりません。

```
1 int main() {
2     // 入力
3     int N, K; cin >> N >> K;
4     vector<vector<int>> A(N + 2, vector<int>(N + 2, 0));
5     for (int i = 1; i <= N; i++) {
6         for (int j = 1; j <= N; j++) {
7             cin >> A[i][j];
8         }
9     }
10
11     // 全探索
12     int Answer = 0;
13     for (int x = 1; x <= N; x++) {
14         for (int y = 1; y <= N; y++) {
15             for (int c = 1; c <= K; c++) {
16                 int colorful = get_colorful(x, y, c, N, A);
17                 Answer = max(Answer, colorful);
18             }
19         }
20     }
21
22     // 出力
23     cout << Answer << endl;
24     return 0;
25 }
```

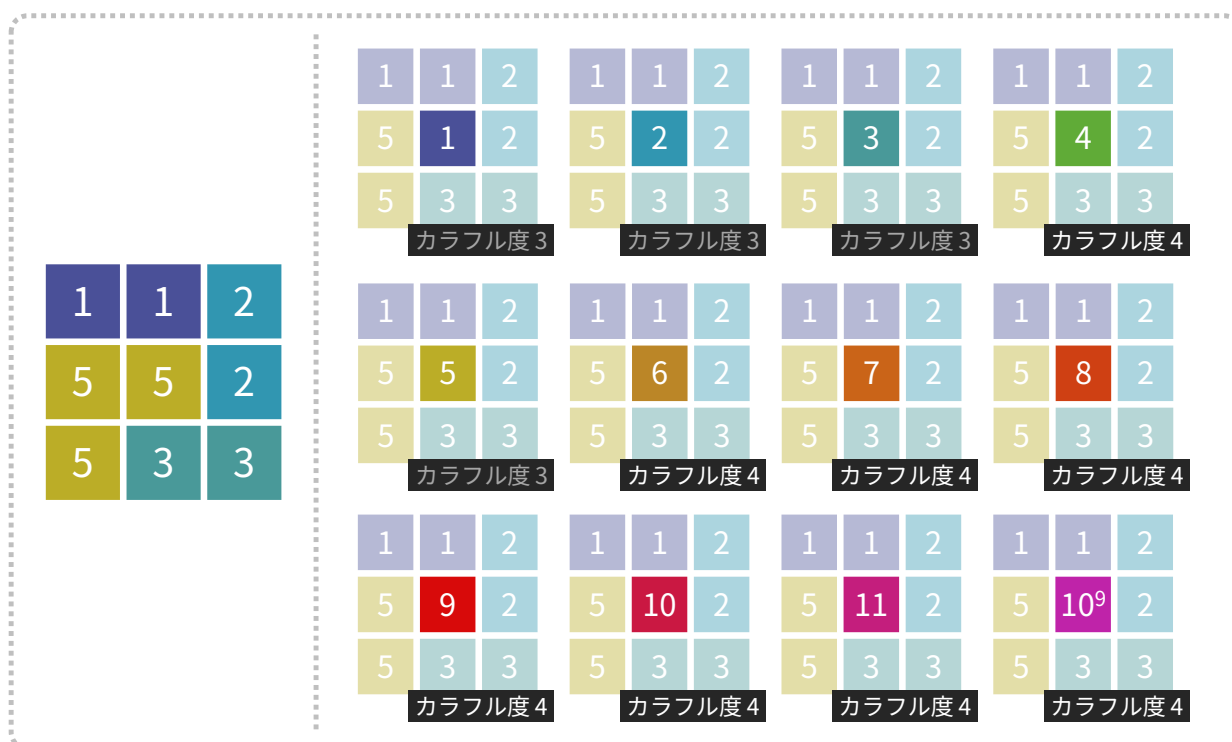


小課題 4 (累計 68 点)

制約: $N \leq 10$

この小課題は K の制約が非常に大きいため、小課題 3 の全探索ではコンピュータでも時間がかかってしまい、不正解となります。具体的に計算してみましょう。この小課題の中で一番大きいケースは、 $N = 10$ および $K = 10^9$ です。全探索では $N^2 \times K$ パターンくらいを試す必要がありますので、 $10^2 \times 10^9 = 10^{11}$ 通りです。ここで競技プログラミングで使われるコンピュータでは、毎秒 $10^8 \sim 10^9$ 回程度の演算しか行えません。実行時間制限は 5 秒であるため、不正解となってしまいます。

そこで次のような考察をします。色 1 から色 10^9 までが使えるとして、果たして色 10^9 を使う意味はあるのでしょうか？たとえば下図左のケースで中央のマスを操作するとします。色 1, 2, 3, 5 に塗り替えた場合のみカラフル度が減ってしまいますが、それ以外の色の場合、カラフル度はすべて同じ 4 となります。このケースの場合、色 10^9 の代わりに色 4 や色 6 を使えばよかったということになります。



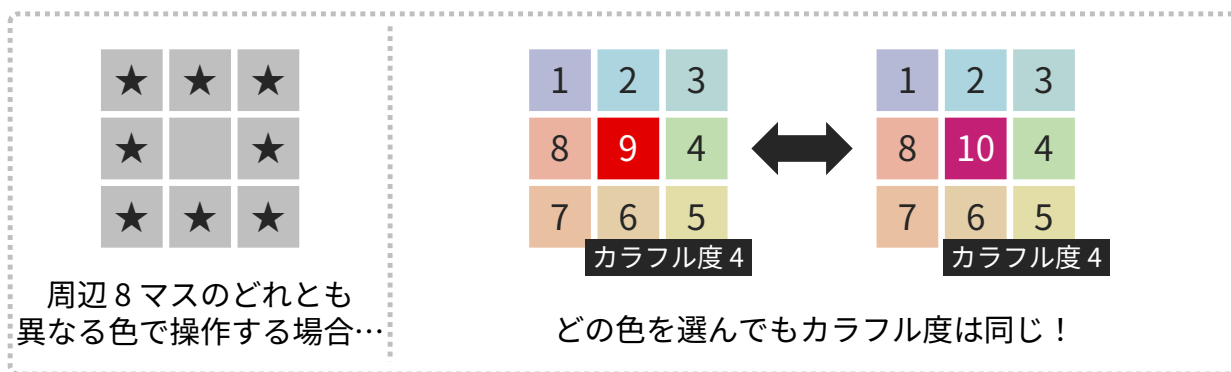
他のケースでも同じなのでしょう。実は、どのようなケースでも、色 1 から色 9 までしか使う意味がないことが証明できます。

この理由を考えてみましょう。操作するマス (x, y) と上下左右斜めに隣り合う最大 8 個のマスの色と、いずれとも異なる色 c で操作するんだと決めたとします。たとえば上の例の色 4 や色 6 が、下図の色 9 や色



10 がそれに該当します。このとき、どのような色 c を選んで操作したとしても、カラフル度は同じです。つまり上の例の場合、色 4 で塗り替えても色 6 で塗り替えてもカラフル度は同じであり、下の例の場合、色 9 で塗り替えても色 10 で塗り替えても答えは同じです。

また、このような色 c が存在する場合、わざわざ周辺の 8 個のマスの色と同じ色で操作するのは、カラフル度を下げただけなので意味がありません。したがって、どのようなケースでも、色 10 以上を使う意味はありません。



そのため、4 ページにある `main` 関数のソースコードの 13-20 行目を次のように修正すると、小課題 4 に正解することができます。

```
1 for (int x = 1; x <= N; x++) {  
2     for (int y = 1; y <= N; y++) {  
3         for (int c = 1; c <= min(K, 9); c++) {  
4             int colorful = get_colorful(x, y, c, N, A);  
5             Answer = max(Answer, colorful);  
6         }  
7     }  
8 }
```

(次ページへ続く)



小課題 5 (累計 100 点)

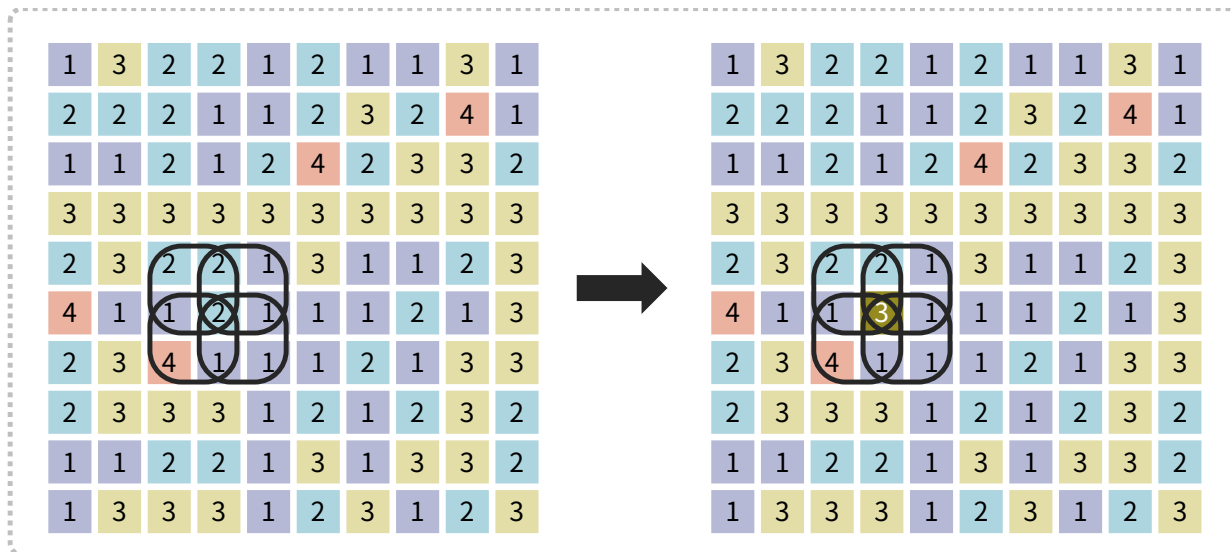
制約：追加制約なし ($N \leq 270, K \leq 10^9$)

小課題 4 の考察をすると、探索すべきパターン数が $N^2 \times K$ 通りから $N^2 \times 9$ 通りに減ります。しかし、現状の実装では各パターンについて、カラフル度を求めるのに N^2 回くらいの計算をしているため、全体の計算回数が $N^4 \times 9$ 回くらいになり、小課題 5 の制約では実行に時間がかかってしまいます。

そこで次のような考察をします。実はカラフル度を直接計算する必要はなく、「操作によってカラフル度がどれだけ変化したか」さえ分かれば、答えが求まる、という考察です。

ここでカラフル度の変化は、操作したマスをも (x, y) とするとき、マス (x, y) を含む最大 4 個の 2×2 の長方形領域さえ見れば、高速に計算することができます。

たとえば以下の図におけるカラフル度の変化を考えてみましょう。右図のカラフル度を手計算で直接求めるのは難しいですが、カラフル度の変化であれば、枠で囲んだ 4 つのうち 3 色以上含んでいるものが 1 個から 3 個に増えたことを使うと、変化がプラス 2 であると簡単に分かります。そのようなアイデアを使うと、計算回数の N^4 の部分が N^2 になり、満点を取ることができます。





3

修学旅行 (School Trip)

Author: 田中 優希 (blackyuki)

0 問題概要

A と B のみからなる長さ 3^N の文字列 S がある. S を連続する 3 文字ずつのグループに分け各グループで多数決をとることで, S を元の $\frac{1}{3}$ の長さの文字列に置き換えるという操作を考える. この操作を N 回繰り返すことで得られる長さ 1 の文字列を求めよ. ただし, 文字列の一字を変更するというクエリを Q 回処理する必要がある.

1 小課題 1 ($N = 1$)

S の 3 文字のうち A と B のどちらが多いかを計算すればよい.

2 小課題 2 ($Q \leq 10$)

操作のシミュレーションを愚直に行えばよい. 計算量は $O(3^N Q)$.

3 小課題 3 ($p_k \leq 5$)

操作前の文字列 S としてありうるものは最大で 2^5 通りであるから, 辞書などのデータ構造を用いてメモ化すると計算量を抑えることができる. 計算量は $O(3^N 2^{\max(p_k)} + Q)$.

4 小課題 4 (T の各文字は A である. また, $p_k \neq p_l$ ($1 \leq k < l \leq Q$).)

答えには単調性がある. すなわち, ある k が存在して, k 番目のクエリまでは答えが A で, それ以降のクエリの答えは B である.

よって, 境界を二分探索することができる. 計算量は $O(3^N \log Q + Q)$.



5 小課題 5 (追加の制約なし)

S に k ($0 \leq k \leq N$) 回の操作を行った後の文字列を S_k とすると、各 k について、1 つのクエリの前後で S_k は高々一文字しか変わらないということが分かる。

よって、前処理として各 S_k を計算した上で、各クエリでは影響を与えうる高々 $N + 1$ 文字のみを再計算することで、この問題を高速に解くことができる。計算量は $O(3^N + NQ)$ 。



4

最悪の記者 5 (Worst Reporter 5)

Author: 平木 康傑 (sheyasutaka)

0 問題概要

$(1, 2, \dots, N)$ の順列 Q が valid であるとは、ある $(1, \dots, N)$ の順列 $(p_1, p_2, \dots, p_N)$ を初期配置として、 $i = 1, 2, \dots, M$ の昇順に以下の操作を失敗なく行うことができ、かつ操作後の順列を Q に一致させられることをいう。

- その時点の順列において A_i と B_i が隣り合っている場合、それらの位置を入れ替える。そうでない場合、操作に失敗する。

valid な順列が存在するか判定し、あれば辞書順最小のものを求めよ。

1 小課題 1 ($N \leq 8, M \leq 8$)

$N!$ 通りの初期配置を全探索することで valid な順列をすべて求め、そのうち辞書順最小のものを求めればよい。時間計算量は $O(N! \cdot NM)$ 。

2 小課題 2 ($A_1, A_2, \dots, A_M, B_1, B_2, \dots, B_M$ は相異なる)

この制約下において、順列 Q は以下の条件を満たすとき、またそのときに限り valid である。

- Q において A_i, B_i は隣接する位置にある。 ($1 \leq i \leq M$)

これは、 Q が以下の計 $N - M$ 個の数列をつなげてできることと等価である。

- 2 要素の連続部分列 (M 個): 各 $i = 1, 2, \dots, M$ について、 (A_i, B_i) または (B_i, A_i)
- 1 要素の連続部分列 ($N - 2M$ 個): $v \notin \{A_1, A_2, \dots, A_M, B_1, B_2, \dots, B_M\}$ を満たす $1 \leq v \leq N$ について、
(v)

このような順列は必ず存在する。



辞書順最小の Q を作るという目標において、以下の 2 つの主張が成り立つ。

1. 2 要素の連続部分列は $(A_i, B_i), (B_i, A_i)$ のうち辞書順で小さいほう (この場合は必ず (A_i, B_i)) を選ぶのが最適である。
2. 上で決定した $N - M$ 個の列をつなげるとき、先頭要素が昇順になるように並べるのが最適である。

したがって、 $N - M$ 個の数列を先頭要素の昇順でソートして、順につなげればよい。

3 小課題 3 ($N \leq 1\,000, M \leq 1\,000$)

valid な順列が存在する場合、順列 Q が valid である条件は、 $1, 2, \dots, N$ を全体でちょうど 1 つずつ含むような数列の集合 $R = \{R_1, R_2, \dots, R_k\}$ を用いて以下のように言い換えられる。

- Q は $R'_1, R'_2, \dots, R'_k$ を任意の並びでつなげて得られる数列である。ここでそれぞれの R'_j は、 R_j に等しいか、 R_j の要素の順序を反転させたものに等しい。

操作が 0 個である状態では、 $R = \{(1), (2), \dots, (N)\}$ とすれば言い換えが成立する。

ここから操作を 1 つずつ追加していくときの R の変化を管理することを考える。

操作 $(A_i, B_i) = (a, b)$ が追加されたとする。

- a, b が同じ列 R_u に属している場合: R_u において a, b が隣接する位置にある場合、単にその位置を入れ替える。そうでない場合、valid な順列は存在しない。
- a, b が異なる列 R_s, R_t に属している場合: a が R_s の先頭または末尾にあり、かつ b が R_t の先頭または末尾にある場合、 R_s, R_t を反転を許してつないだ新しい列 R_u で置き換えた後、 a, b の位置を入れ替える。そうでない場合、valid な順列は存在しない。
 - 新しい列 R_u の構成: a, b が隣り合うように、 R'_s, R'_t を任意の順につなげればよい。たとえば、 a が R_s の先頭、 b が R_t の先頭だった場合、 $R_u := \text{rev}(R_t) + R_s$ のように構成できる。

たとえば、 $R = \{(1, 2, 3), (4, 5, 6), (7, 8, 9)\}$ の状態から操作 $(3, 6)$ が追加されたとき、新しい R は $R = \{(1, 2, 6, 3, 5, 4), (7, 8, 9)\}$ となる。

最終的な R を得たあとは、以下にしたがって列を並べればよい。

1. 各列の向きは、 $R_j, \text{rev}(R_j)$ のうち辞書順で小さいほうを選ぶのが最適である。
2. 決定した列をつなげるとき、先頭要素が昇順になるように並べるのが最適である。

これらは愚直に実装しても時間計算量 $O(NM)$ で行える。



4 小課題 4 (i 回目 ($2 \leq i \leq M$) の順位変動において, A_i と B_i のうち少なくとも片方は $A_1, A_2, \dots, A_{i-1}, B_1, B_2, \dots, B_{i-1}$ の中に現れない値である)

この制約下において, 操作で与えられる 2 点のうち少なくとも片方は, 必ず長さ 1 の列に属する.

追加する操作 (a, b) において b が長さ 1 の列に属するとする. このとき, a が列の先頭であれば先頭から, 末尾であれば末尾から以下の操作を行えばよい.

1. a を削除する.
2. b を追加する.
3. a を追加する.

たとえば, $R_j = (1, 2, 3, 4)$ に対して操作 $(1, 6)$ を追加するとき, 新しい R_j は $R_j = (1, 6, 2, 3, 4)$ となる.

これは両端キュー (deque) と呼ばれるデータ構造によって実現できる. 1 つの deque は 2 つの queue の組合せで実装できる.

ある値がどの列に属するかを管理することで, 時間計算量を $O(N + M)$ にできる.

5 小課題 5 (追加の制約なし)

5.1 解法 1: 重軽マージ

小課題 3 の解法を deque で実装することを考えると, 列 R_s, R_t をつなぐ操作は, 列 R_s に R_t の各要素を push することで実現できるので, 計算量は R_t 側の要素数のみで抑えられる. そこで, 常に要素数が少ないほうを push する側 (R_t) におくと, 実は全体での push 回数は $O(N \log N)$ のオーダーに改善される.

証明: 任意の要素 x について, x が push されるごとに x の属する列の長さは 2 倍以上になっている. したがって, x が push される回数は $\log_2 N$ 以下である.

ある値がどの列のどの位置に属するかを管理することで, 時間計算量を $O(M + N \log N)$ にできる.

5.2 解法 2: 双方向リスト

双方向リストを用いて同様の解法を実装すると, 列をつなげる操作が定数時間で実現できる.

2 つの値が同じ列に属するかを判定するために, 追加の情報を持つ必要があることに注意が必要である. これは Union-Find 木によって実現できる.

時間計算量は $O(N + M\alpha(N))$ である. ここで, $\alpha(N)$ はアッカーマンの逆関数である.



5

神経衰弱 2 (Pair Matching 2)

Author: 山縣 龍人 (tatyam)

0 問題概要

$1, 2, \dots, N$ の書かれたカードが 2 枚ずつ、合計 $2N$ 枚のカードが一行に並んでいる。ビ太郎は左から順にカードを見ていき、それぞれ拾うか拾わないかを選ぶ。「拾ったカードに書かれていた整数」が、「1 つ前に拾ったカードに書かれていた整数」または「2 つ前に拾ったカードに書かれていた整数」と一致しているとき、拾ったカードに書かれていた整数を a として V_a 点を得る。ビ太郎が得られる最大の得点はいくつか？

1 考察

最終的な得点に関わらないカードは拾っても無駄であるから、そのようなカードは拾わないものとする。この制約を加えたとき、あなたは以下のようにカードを拾う必要がある。

- カード i と同じ整数が書かれたもう 1 枚のカードをカード p_i とする。
- カード i を拾うならば、カード p_i も拾わなければならない。さらに、カード i とカード p_i の間で拾うカードは 1 枚以下である。

この制約の中で、どのようにカードを拾うことができるかを考える。

整数 x が書かれている 2 枚のカードをペア x と呼ぶことにする。また、ペアの 2 枚のカードの間の区間 (両端を含まない) を区間 x と呼ぶ。

- 例えば、 $A = (1, 1, 2, 2, 3, 3)$ と並んだカードをすべて拾おうとすることは、各ペアの作る区間にカードが存在しないから、可能である。
- $A = (1, 2, 2, 1)$ と並んだカードをすべて拾おうとすることは、区間 1 にカードが 2 枚存在するから、不可能である。
- $A = (1, 2, 1, 2)$ と並んだカードをすべて拾おうとすることは、各区間に存在するカードが 1 枚であるから、可能である。

これをまとめると、拾うペア (区間) を集めた集合 X について、以下のことがわかる。



1. (包含) X の 2 つのペア a, b が, それらのみを取り出すと a, b, b, a の順に並んでいることはない.
2. (交差) X の 2 つのペア a, b が, それらのみを取り出すと a, b, a, b の順に並んでいるものとし, カードの番号をそれぞれ a_1, b_1, a_2, b_2 とする. カード a_1 とカード b_2 の間からはこれ以外のカードを拾うことはできない. さらに, カード a_1, b_1, a_2, b_2 が間にあるようなペアも拾うことができない. したがって, 区間 $a \cup b$ は他の区間から独立している.
3. 条件 1. より, 区間 a と交差する区間が存在しないならば, 区間 a は他の区間から独立している.

したがって, X は,

- X の他の区間と独立な区間 a
- 区間 a, b が交差していて, 区間 $a \cup b$ が X の他の区間と独立であるような区間 a, b

を集めたものである.

2 小課題 3 ($N \leq 9$)

どのカードを拾うかの 2^{2N} 通りを全探索し, ビ太郎の動きをシミュレーションすればよい. 計算量は $O(2^{2N} \cdot N)$ 時間である.

3 小課題 4 ($N \leq 18$)

どのペアを拾うかの 2^N 通りを全探索し, ビ太郎の動きをシミュレーションすればよい. 計算量は $O(2^N \cdot N)$ 時間である.

4 小課題 1 ($(A_1, \dots, A_N) = (1, \dots, N), N \leq 5\,000$)

上の考察により, 3 つ以上のペアを拾うことができないことがわかる.

1 個または 2 個のペアの選び方 $\frac{N(N+1)}{2}$ 通りを全探索し, それらを拾うことが可能かを判定する. 計算量は $O(N^2)$ 時間である.

5 小課題 2 ($(A_1, \dots, A_N) = (1, \dots, N)$)

ペア A_i とペア A_j ($N+1 \leq i < j \leq 2N$) は, $A_i < A_j$ であるとき同時に拾うことができる. したがって, j を固定したときに選ぶべき i は, $N+1 \leq i < j$ かつ $A_i < A_j$ を満たす i のなかで V_{A_i} が最大のものである.



j の昇順に走査すると, $\max\{V_{A_i} \mid N+1 \leq i < j, A_i < A_j\}$ を求めることは 1 点更新・区間最大値取得のセグメント木で可能である. 計算量は $O(N \log N)$ 時間である.

長さ N の 1 点更新・区間最大値取得のセグメント木 seg を用意する.

$t = N+1, \dots, 2N$ の順に走査する. 各 $N+1 \leq i \leq 2N$ について, $i < t$ である間 $seg[A_i] = V_{A_i}$ とする.

(すなわち, $t = i$ のステップが終わるときに $seg[A_i]$ に V_{A_i} を代入する.)

$\max\{V_{A_i} \mid N+1 \leq i < j, A_i < A_j\}$ の値は, $t = j$ であるときの $\max(seg[1 : A_j])$ の値である.

6 小課題 5 ($N \leq 300$)

$dp[\text{カード } i \text{ まで見た}][\text{左手に持ったカード}][\text{右手に持ったカード}] = (\text{これまでに得られる得点の最大値})$ の動的計画法を行う. $O(N^3)$ 状態で, 遷移が $O(1)$ であるから, 計算量は $O(N^3)$ 時間である. DP の更新の方法については, 小課題 6 を参考にせよ.

7 小課題 6 ($N \leq 3000$)

上の DP の [カード i まで見た] の次元をなくし, in-place に行うことを考える. カード i を追加するとき, 以下の更新が起こる.

- 各 $1 \leq y \leq N$ について, $dp[A_i][y] \leftarrow \left(\max_{1 \leq x \leq N} dp[y][x] \right)$
- 各 $1 \leq y \leq N$ について, $dp[A_i][y] \leftarrow dp[y][A_i] + V_{A_i}$
- $dp[A_i][A_i] \leftarrow \left(\max_{1 \leq x \leq N} dp[A_i][x] \right) + V_{A_i}$

$dp[y][x]$ の配列とともに, $mx[y] = \max_{1 \leq x \leq N} dp[y][x]$ の配列を管理することで, カードの追加を $O(N)$ で行うことができる. 計算量は $O(N^2)$ 時間である.

8 小課題 7 ($V_i = 1, N \leq 150\,000$)

ペア i の左のカードをカード l_i , 右のカードをカード r_i とする. 各ペア i に対して, $l_j < l_i < r_j < r_i$ を満たすペア j が存在するならば, そのような l_j の最大値を L_i と定義する. L_i は, 1 点変更・区間最大値取得のセグメント木で求めることができる.



長さ $2N$ の 1 点変更・区間最大値取得のセグメント木 seg を用意し、カードを $t = 1, \dots, 2N$ の順に走査する.

$l_i < t < r_i$ の間, $seg[r_i] = l_i$ とする. それ以外のときは, $seg[r_i] = -\infty$ とする.

このとき, L_i は $t = l_i$ のときの $\max(seg[l_i : r_i])$ の値である. $\max(seg[l_i : r_i]) = -\infty$ ならば, L_i は存在しない.

最初の考察をもとにすると, 問題は以下に帰着される.

以下の $2N$ 個程度の区間から, 互いに独立になるように区間をいくつか選んだときの, 選んだ区間の重みの総和としてあり得る最大値を求めよ.

- 各 $1 \leq i \leq N$ について, 重み 1 の区間 (l_i, r_i) .
- 各 $1 \leq i \leq N$ について, 重み 2 の区間 (L_i, r_i) .

これは, いわゆる「区間スケジューリング問題」を少し発展させたものである.

区間スケジューリング問題では, 区間を右端の昇順でソートし,

- 区間 $1, \dots, i$ から互いに独立になるように区間をいくつか選ぶときの最大個数 x
- x 個の区間を互いに独立になるように選ぶときの, 右端の座標の最小値 r_0

を管理する. これを発展させて,

- 区間 $1, \dots, i$ から互いに独立になるように区間をいくつか選ぶときの最大重み x
- 合計重み x の区間を互いに独立になるように選ぶときの, 右端の座標の最小値 r_0
- 合計重み $x - 1$ の区間を互いに独立になるように選ぶときの, 右端の座標の最小値 r_{-1}

を管理することで, 上の問題を解くことができる. 計算量は $O(N \log N)$ 時間である.

重み付き区間スケジューリング問題

また, 重み付き区間スケジューリング問題として, 以下のように解くこともできる.



長さ $2N$ の 1 点更新・区間最大値取得のセグメント木 dp を用意する.

$dp[t] = (\text{カード } 1, \dots, t-1 \text{ からいくつか拾い, カード } t \text{ を必ず拾うときの得点の最大値})$ となるように計算する.

区間を右端の昇順でソートし, 順に走査する.

重みが w の区間 (l, r) が来たとき, $dp[r] \leftarrow \max(seg[1 : l]) + w$ で更新する.

最終的な $\max(dp)$ が答えである.

9 満点 ($N \leq 400\,000$)

小課題 7 における, L_i を求める方法と, 重み付き区間スケジューリング問題の解法を応用して, 以下のよう
に解くことができる. 計算量は $O(N \log N)$ 時間である.

長さ $2N$ の 1 点更新・区間最大値取得のセグメント木 $dp1, dp2$ を用意し, $t = 1, \dots, 2N$ の順に走査
する.

$dp1$ は, $dp[t] = (\text{カード } 1, \dots, t-1 \text{ からいくつか拾い, カード } t \text{ を必ず拾うときの得点の最大値})$ とな
るように計算する.

$dp2$ は, 各ペア (l_i, r_i) について, $l_i < t < r_i$ である間, $dp2[r_i] = \max(dp1[1 : l_i]) + V_i$ とする. それ以
外のときは, $dp2[r_i] = -\infty$ とする.

$t = 1, \dots, 2N$ の順に, $t = r_i$ である区間 (l_i, r_i) があるとき,

- $dp1[r_i] \leftarrow \max(dp1[1 : l_i]) + V_i$
- $dp1[r_i] \leftarrow \max(dp2[l_i : r_i]) + V_i$

で更新する.

最終的な $\max(dp1)$ が答えである.